

74. Nommer un geste qu'on refait

Une fonction avec un argument, et trois appels.

1. Ce qu'il faut comprendre

Les fonctions prennent des arguments, et c'est là tout l'intérêt. Sans argument, il faudrait trois fonctions, ou une variable globale à poser avant chaque appel.

`void` dit qu'elle ne rend rien. Le compilateur le vérifie : une fonction qui annonce rendre un `uint8_t` sans jamais faire `return` est **refusée, avec sa ligne**.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **232 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
rangee()       32 o ← main() - ta fonction
main()         28 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AdresseCase    10 o ← poser() - trouve l'adresse d'une case de l'écran
CopierTuiles   31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         16 o ← CopierTuiles - 0 lettre sur 43, et 1 tuile dessinée

Lettres gravées : aucune
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond,
EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :
poser() × 1    13 o sur place, dans rangee() + AdresseCase (10 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :
rangee() × 3   21 o sur place, dans main()
```

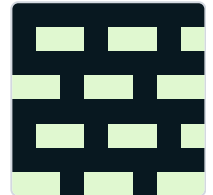
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : dessine trois rangées de briques avec UNE fonction, appelée 3 fois.
//
// Une FONCTION donne un nom à un groupe d'instructions.
// void rangee(uint8_t ligne) ...
// void      = elle ne rend pas de résultat
// rangee    = son nom
// uint8_t ligne = ce qu'on lui donne à l'appel (un « argument »)
// rangee(4); → exécute la fonction avec ligne = 4.

#include <Tuile> // un dessin de 8 × 8 pixels
#include <poser> // pose une tuile sur une case du fond
```

2. UN DESSIN — BRIQUE

```
Tuile BRIQUE = {
    "#####", "#..#..#", "#####", "..#..#..",
    "#####", "#..#..#", "#####", "..#..#..",
};
```



à l'écran

3. UNE FONCTION — RANGEE()

```
void rangee(uint8_t ligne) { // Une rangée de briques...
    for (uint8_t x = 0; x < 20; x++) poser(x, ligne, BRIQUE); // ... sur les 20 colonnes
} // de la ligne demandée.
```

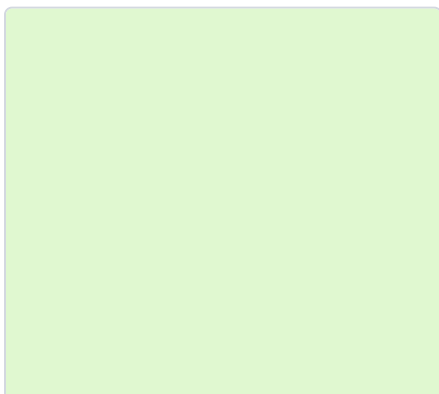
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() { // Le jeu commence ici.
    rangee(4); // Une rangée sur la ligne 4,
    rangee(9); // une sur la ligne 9,
    rangee(14); // une sur la ligne 14.

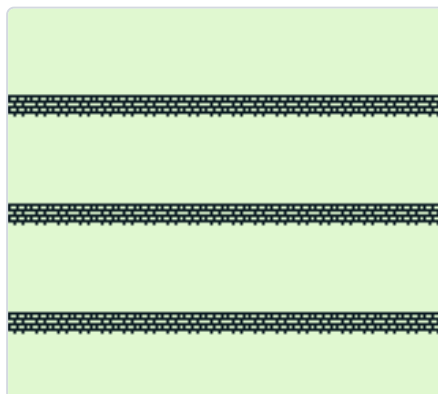
    while (true) { // Répète pour toujours :
        image(); // attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Trois rangées de briques, écrites par un seul bout de code.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la première rangée est posée

✓ la deuxième aussi

✓ et la troisième

✓ rien entre les rangées

6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

urs et fonctions, une seule suite pour écrire une cartouche Game Boy en C++

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

BRIQUE

+ tuile 8 × 8

+ perso 16 × 16

+ perso 32 × 32

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

BRIQUE

tuile n° 44 — poser(x, y, BRIQUE)

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

×2

÷2

Agrandir...

Vider le dessin

Taille voulue :

✓ Créer la copie

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

BRIQUE ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

BRIQUE peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre... ▼

Ensuite : **75. Une fonction qui rend une valeur** — chapitre 11 : Découper son programme. · gameboy3 — écrire une cartouche Game Boy en C++